今天和接下来几堂课的内容

- 开始微架构
- 微架构--单周期CPU设计
- 微架构--多周期CPU设计
- 流水线
- ...

回顾：RISC vs CISC

- 通常来说：

- RISC
  - 简单的指令
  - 固定长度
  - 统一解码
  - 少量的寻址模式
- CISC
  - 复杂的指令
  - 可变长度
  - 非统一解码
  - 多种寻址模式

现在我们有了一个ISA

- **我们如何实现它？**
  - 即，我们如何设计一个符合硬件/软件接口的系统？
- **题外话：“系统”可以是单独的硬件，也可以是硬件和软件的组合**
  - 记住“ISA翻译” （ Translation of ISAs ）
  - 一个虚拟ISA可以通过“软件”转换为实现ISA
- **我们大多数时候将假设是“硬件”**

今天的内容

- 实现ISA：微架构基础

计算机是如何处理指令的？

- 计算机是如何处理指令的？
- 处理指令意味着什么？

- 冯·诺依曼模型
  1. AS = 指令处理前的架构（程序员可见）状态
  2. 处理指令
  3. AS’ = 指令处理后的架构（程序员可见）状态

- 处理指令：根据指令的ISA规范，将 AS 转换为 AS’。

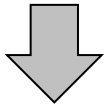
“处理指令”步骤

- **ISA 抽象地规定了给定指令和 AS（架构状态）之间 AS’ 应该是什么样的**
  - 它定义了一个抽象的有限状态机，其中：
    - **状态** = 程序员可见的状态
    - **下一状态逻辑** = 指令执行规范
- **从 ISA 的角度来看，在指令执行过程中，AS 和 AS’ 之间没有“中间状态”**
  - 每条指令只经历一次状态转换
- **微架构 实现了如何将 AS 转换为 AS’**
  - 在实现中有很多选择
  - 我们可以使用程序员不可见的状态来优化指令执行的速度，实现多次状态转换
    - 选择1：**AS → AS’**（在一个时钟周期内完成转换）
    - 选择2：**AS → AS+MS1 → AS+MS2 → AS+MS3 → AS’**（多个时钟周期来完成转换）

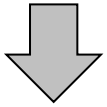
一个非常基础的指令处理引擎

- 一个非常基础的指令处理引擎
- 每条指令执行只需要一个时钟周期
  - 只使用组合逻辑来实现指令执行
    - 没有中间的、程序员不可见的状态更新

AS = 时钟周期开始时的架构（程序员可见）状态



Process instruction in one clock cycle



AS’ = 时钟周期结束时的架构（程序员可见）状态

组合逻辑（Combinational Logic）

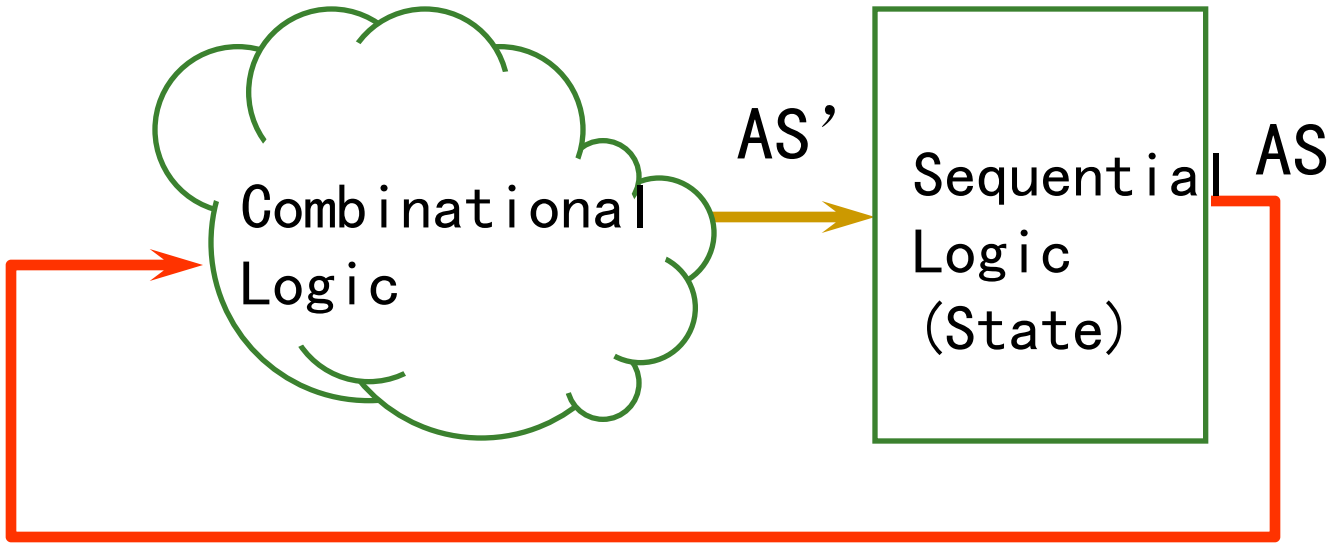
- 数字电路设计中的一种基本类型，输出仅依赖于当前输入状态的电路。
- 没有存储功能
- 输出由输入决定
- 无时钟控制

- 加法器，逻辑门，多路选择器。

- 在计算机的微架构设计中，组合逻辑用于实现各种基本运算和信号传输任务。例如：
  - 算术运算：执行加法、减法、乘法等运算。
  - 数据传输：将数据从一个寄存器传送到另一个寄存器。
  - 地址解码：选择要访问的内存地址。

一个非常基础的指令处理引擎

- 单周期机器



- 时钟周期时间由什么决定？
- 组合逻辑的关键路径由什么决定？

单周期机器 vs. 多周期机器

- **单周期机器**
  - 每条指令执行只需要一个时钟周期
  - 所有状态更新都在指令执行结束时进行
  - **主要缺点**：最慢的指令决定了时钟周期时间 → 导致较长的时钟周期
- **多周期机器**
  - 指令处理被拆分成多个周期/阶段
  - 在指令执行期间，可以进行状态更新
  - 只有在指令执行结束时才会进行架构状态更新
  - 相比单周期机器的优点：最慢的“阶段”决定了时钟周期时间
- **单周期和多周期机器 都严格遵循冯·诺依曼模型（von Neumann model）在微架构层面上的实现**

指令处理“周期”

- 指令在“控制单元”的指导下，一步步进行处理。
- 指令周期：处理一条指令的步骤序列。
- 基本上，指令处理分为六个阶段：
  - Fetch：获取指令
  - Decode：解码指令
  - Evaluate Address：计算地址
  - Fetch Operands：获取操作数
  - Execute：执行
  - Store Result：存储结果
- 并非所有指令都需要经历所有六个阶段。

指令处理“周期” vs. 机器时钟周期

- **单周期机器：**
  - 指令处理周期的所有六个阶段需要 **一个机器时钟周期** 来完成。
- **多周期机器：**
  - 指令处理周期的所有六个阶段需要 **多个机器时钟周期** 来完成。
  - 事实上，**每个阶段可能需要多个时钟周期** 来完成。

单周期机器 vs. 多周期机器：控制与数据

- 单周期机器 vs. 多周期机器：控制与数据
- 单周期机器：
  - 控制信号与数据操作信号在同一时钟周期内生成。
  - 每条指令的所有操作都在一个时钟周期内完成（串行处理）。
- 多周期机器：
  - 下一周期所需的控制信号可以在当前周期生成。
  - 控制处理的延迟可以与数据路径操作的延迟重叠（更多的并行性）。
- 我们将在 微程序化多周期微架构 中清楚地看到这个区别。

数据路径和控制设计的多种方式

- 设计数据路径和控制逻辑的方式有很多种。

- 单周期、多周期、管道化的数据路径和控制
- 单总线与多总线的数据路径
- 硬连线/组合逻辑与微编码/微程序控制
  - 控制信号由组合逻辑生成
  - 控制信号存储在内存结构中

- 控制信号和结构依赖于数据路径设计

性能分析

- **指令的执行时间**
  - {CPI} × {时钟周期时间}
- **程序的执行时间**
  - 所有指令的总和 [{CPI} × {时钟周期时间}]
  - {指令数} × {平均CPI} × {时钟周期时间}
- **单周期微架构性能**
  - CPI = 1
  - 时钟周期时间长
- **多周期微架构性能**
  - 每条指令的CPI不同
  - 平均CPI → 希望较小
  - 时钟周期时间短

下节课的内容

- 微架构--单周期CPU设计
- 微架构--多周期CPU设计
- 流水线
- ...